theorem

Vera: Weakest Preconditions for TICL over Rust

From Imp Structural Lemmas to Rust VCGen Rules

Lef Ioannidis

2026

Background: TICL over Imp

TICL Structural Lemmas for Imp

Core idea (from the TICL paper)
For a small imperative language `StImp` with mutable state, the entailment $[t,\ m \Vdash_L\ \varphi]_S$ is proved structurally via inference rules on the syntax of t.

Key rules from `imp-lemmas.tex`:

- **Seq**$_S$AU$_L$: sequential composition decomposes into bind
- **If**$_S$AU$_L$: conditional branches both satisfy the obligation
- **While**$_S$AG: infinite loop with coinductive invariant $\mathcal{R}$
- **While**$_S$AU$_L$: terminating loop with ranking function f

Goal: Lift these to Rust syntax $\rightarrow$ define `wp(_, _)` for Vera.

wp(Rust, TICL) — Weakest Preconditions

Notation

$$\sigma \in \text{State (mutable references, heap)}$$

$$[t,\ \sigma \Vdash_L\ \varphi]_{\texttt{Rust}} \triangleq \text{“Rust term } t \text{ from state } \sigma \text{ satisfies } \varphi\text{”}$$

$$\texttt{wp}(t,\ \varphi) \triangleq \text{weakest precondition on } \sigma \text{ s.t. } [t,\ \sigma \Vdash_L\ \varphi]_{\texttt{Rust}}$$

Temporal formulas

- $\varphi ::= \texttt{now}\ P \mid \texttt{AG}\ \varphi \mid \varphi\ \texttt{AU}\ \varphi' \mid \varphi \wedge \varphi'$
- Sugar: $\texttt{AF}\ \varphi' = \top\ \texttt{AU}\ \varphi', \quad \texttt{AX}\ \varphi' = \top\ \texttt{AN}\ \varphi'$
- $\texttt{done}\ Q$: termination postcondition; $\texttt{now}\ P$: intermediate state predicate

Rule: Assignment

Rust: $*x = e$

$$\frac{\sigma' = \sigma[x \mapsto v] \qquad \varphi(\sigma') \qquad \varphi'(\sigma') \vee \text{(continue checking)}}{[*x = e,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{Assign}_{\textsc{Rust}}\textsc{AU}_L$$

Weakest precondition

$$\texttt{wp}(*x = e,\ \varphi\ \texttt{AU}\ \varphi') \ = \ \varphi[\sigma'/\sigma] \wedge (\varphi'[\sigma'/\sigma] \vee \top)$$

where $\sigma' = \sigma[x \mapsto e_\sigma]$

Analogous to **Assign** in Imp: $s \leftarrow x_s$ updates the state map.

Rule: Sequential Composition (let-binding)

Rust: $\texttt{let}\ x = t;\ u$

$$\frac{[t,\ \sigma \Vdash_R\ \ \varphi\ \texttt{AU AX done}\ \mathcal{R}]_{\texttt{Rust}} \qquad \forall\ x, \sigma',\ \mathcal{R}\ x\ \sigma' \to [u[x],\ \sigma' \Vdash_L\ \ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}{[\texttt{let}\ x = t;\ u,\ \sigma \Vdash_L\ \ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{Let}_{\textsc{Rust}}\textsc{AU}_L$$

Weakest precondition

$$\texttt{wp}(\texttt{let}\ x = t;\ u,\ \varphi\ \texttt{AU}\ \varphi') \ =\ \texttt{wp}(t,\ \varphi\ \texttt{AU AX done}\ \mathcal{R}) \wedge \forall x, \sigma'.\ \mathcal{R}\ x\ \sigma' \to \texttt{wp}(u[x],\ \varphi\ \texttt{AU}\ \varphi')$$

Direct lift of **BindAU**$_L$ / **Seq**$_S$AU$_L$ from TICL.

Rule: Conditional

Rust: $\texttt{if } c \, \{t\} \, \texttt{else} \, \{u\}$

$$\frac{\begin{cases} [c,\ \sigma \Vdash_R\ \texttt{AX done}_{=}\ (b, \sigma')]_{\texttt{Rust}} \\ [t,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}, & \text{if } b \\ [u,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}, & \text{otherwise} \end{cases}}{[\texttt{if } c \, \{t\} \, \texttt{else} \, \{u\},\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{If}_{\textsc{Rust}}\textsc{AU}_L$$

Weakest precondition

$$\texttt{wp}(\texttt{if } c \, \{t\} \, \texttt{else} \, \{u\},\ \varphi\ \texttt{AU}\ \varphi') \ =\ \texttt{wp}(c,\ \texttt{AX}\, \texttt{done}_{=}\ (b, \sigma')) \wedge \begin{cases} \texttt{wp}(t,\ \varphi\ \texttt{AU}\ \varphi')[\sigma'/\sigma], & \text{if } b \\ \texttt{wp}(u,\ \varphi\ \texttt{AU}\ \varphi')[\sigma'/\sigma], & \text{othe} \end{cases}$$

Evaluating c takes steps that must satisfy the path property φ. Direct lift of $\mathbf{If}_S\mathsf{AU}_L$.

Rule: While Loop (AU — terminating)

Rust: `while` c $\{t\}$ **with** `invariant` $\mathcal{R}$, `decreases` f

$$\frac{\begin{array}{l}\forall\ \sigma,\ \mathcal{R}\ \sigma \to [c,\ \sigma \Vdash_R\ \texttt{AX done}_{=}\ (b, \sigma)]_{\texttt{Rust}} \qquad \mathcal{R}\ \sigma \\ \qquad \wedge \begin{cases} [t,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}} \vee & \\ \quad [t,\ \sigma \Vdash_R\ \varphi\ \texttt{AU AX done}\ (\lambda\ \sigma' \Rightarrow \mathcal{R}\ \sigma' \wedge f\ \sigma' < f\ \sigma)]_{\texttt{Rust}}, & \text{if } b \\ \varphi'(\sigma), & \text{otherwise} \end{cases}\end{array}}{[\texttt{while}\ c\ \{t\},\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']_{\texttt{Rust}}}\ \textsc{While}_{\textsc{Rust}}\textsc{AU}_L$$

Direct lift of $\textbf{While}_S\textsf{AU}_L$. The `decreases` annotation provides f.

Rule: Loop (AG — infinite)

Rust: `loop {t}` **with** `invariant` $\mathcal{R}$ **(no** `decreases`**)**

$$\frac{\begin{array}{l}\mathcal{R}\ \sigma \\ \forall\ \sigma,\ \mathcal{R}\ \sigma \rightarrow \\ \quad [\texttt{loop}\ \{t\},\ \sigma \Vdash_{L}\ \varphi]_{\texttt{Rust}} \wedge \\ \quad [t,\ \sigma \Vdash_{R}\ \texttt{AX}\ (\varphi\ \texttt{AU}\ \texttt{AX done}\ (\lambda\ \sigma' \Rightarrow \mathcal{R}\ \sigma'))]_{\texttt{Rust}}\end{array}}{[\texttt{loop}\ \{t\},\ \sigma \Vdash_{L}\ \texttt{AG}\ \varphi]_{\texttt{Rust}}}\ \textsc{Loop}_{\textsc{Rust}}\textsc{AG}$$

Direct lift of **While**$_S$AG / **IterAG**. Infinite loop = no exit condition. The coinductive hypothesis $[\texttt{loop}\ \{t\},\ \sigma \Vdash_{L}\ \varphi]_{\texttt{Rust}}$ enables proving the property holds at the loop entry after each iteration.

Function Calls and Async/Await

Rule: Function Call (Bind Rule)

Rust: $\texttt{let } x = f(); \; k \; x$

$$\frac{\begin{array}{c} f \texttt{ ensures AF done } \mathcal{R} \\ \forall\, x, \sigma',\ \mathcal{R}\ x\ \sigma' \to [k\ x,\ \sigma' \Vdash_L\ \varphi \texttt{ AU } \varphi']_{\texttt{Rust}} \end{array}}{[\texttt{let } x = f(); \; k\ x,\ \sigma \Vdash_L\ \varphi \texttt{ AU } \varphi']_{\texttt{Rust}}}\ \textsc{Call}_{\textsc{Rust}}\textsc{AU}_L$$

Contract-based transparent checking (Vera extension)
If f has temporal ensures Ψ, additionally assert $\Psi \implies \Phi$ where Φ is the caller's temporal obligation. This ensures f's intermediate states satisfy the caller's properties.

Rule: Async Bind (Lazy Future Creation)

Rust: $\texttt{let}\ p = \texttt{async}\ f();\ k\ p$

$$\frac{\begin{array}{c}\text{No state change (lazy future)} \\ [k\ p,\ \sigma \Vdash_L\ \Phi]_{\texttt{Rust}}\end{array}}{[\texttt{let}\ p = \texttt{async}\ f();\ k\ p,\ \sigma \Vdash_L\ \Phi]_{\texttt{Rust}}}\ \textsc{AsyncBind}_{\textsc{Rust}}$$

Rust futures are lazy: `async fn()` returns a handle, body runs at `.await`. The temporal obligation Φ passes through to the continuation k unchanged.

Rule: Await (AU/AF — terminating callee)

Rust: $\texttt{let}\ x = p.\texttt{await};\ k\ x$

$$\frac{\begin{array}{c} P[p]\ \texttt{ensures}\ \varphi'\ \texttt{AU AX done}\ \mathcal{R} \\ \varphi' \implies \varphi \quad \text{(transparent)} \\ \forall\ x, \sigma',\ \mathcal{R}\ x\ \sigma' \to [k\ x,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'']_{\texttt{Rust}} \end{array}}{[\texttt{let}\ x = p.\texttt{await};\ k\ x,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'']_{\texttt{Rust}}}\ \textsc{Await}_{\textsc{Rust}}\textsc{AU}_L$$

- $P[p]$ is the async process behind future p
- The callee's path property φ' must imply the caller's φ (transparent await)
- After await returns with $\mathcal{R}$, continuation k continues the obligation

Rule: Await (AG — diverging callee)

Rust: $p.\texttt{await}$ **where** $P[p]$ $\texttt{ensures AG}$ ψ

$$\frac{P[p]\ \texttt{ensures AG}\ \psi \qquad \psi \implies \varphi}{[p.\texttt{await},\ \sigma \Vdash_L\ \texttt{AG}\ \varphi]_{\texttt{Rust}}}\ \textsc{Await}_{\textsc{Rust}}\textsc{AG}$$

- If the awaited process has $\texttt{AG}\ \psi$, it runs forever
- The await point becomes a *divergence point*
- The caller's $\texttt{AG}\ \varphi$ is discharged if $\psi \implies \varphi$
- Continuation after await is unreachable (dead code)

Summary

wp(Rust, TICL) — Complete Rule Set

Rust Construct	TICL Rule	Imp Analog
$*x = e$	State update, check φ	Assign
`let` $x = t$; u	BindAU$_L$	Seq$_S$AU$_L$
`if` c $\{t\}$ `else` $\{u\}$	Case split	If$_S$AU$_L$
`while` c $\{t\}$ + `decreases`	IterAU$_L$	While$_S$AU$_L$
`loop` $\{t\}$ (no exit)	IterAG	While$_S$AG
$f()$	Bind + transparent	BindAU$_L$
`let` $p =$ `async` $f()$; k	No-op (lazy)	—
p`.await` (AU callee)	Bind + implication	BindAU$_L$
p`.await` (AG callee)	Divergence	—

Key extension over Imp: contract-based transparent temporal checking at function calls and `.await` boundaries.